

Experiment No. 4

Aim: To study and implement data preprocessing in NLP

Theory:

Natural language processing (NLP) refers to the branch of computer science—and more specifically, the branch of artificial intelligence or AI—concerned with giving computers the ability to understand text and spoken words in much the same way human beings can. The essence of Natural Language Processing lies in making computers understand the natural language. That's not an easy task though. Computers can understand the structured form of data like spreadsheets and the tables in the database, but human languages, texts, and voices form an unstructured category of data, and it gets difficult for the computer to understand it, and there arises the need for Natural Language Processing. There's a lot of natural language data out there in various forms and it would get very easy if computers can understand and process that data. We can train the models in accordance with expected output in different ways.

Breaking a complex problem into a number of small problems, making models for each of them and then integrating these models. A similar thing is done in NLP. We can break down the process of understanding English for a model into a number of small pieces. Natural Language Processing (NLP) is a subfield of computer science and artificial intelligence that deals with the interaction between computers and human languages. The primary goal of NLP is to enable computers to understand, interpret, and generate natural language, the way humans do.

NLP involves a variety of techniques, including computational linguistics, machine learning, and statistical modelling. These techniques are used to analyse, understand, and manipulate human language data, including text, speech, and other forms of communication.

Some of the main applications of NLP include language translation, speech recognition, sentiment analysis, text classification, and information retrieval. NLP is used in a wide range of industries, including finance, healthcare, education, and entertainment, to name a few.

There are three main components:

1. **Tokenization:** Tokenization is about splitting strings of text into smaller pieces, or “Tokens”. Paragraphs can be tokenized into sentences and sentences can be tokenized into words.
2. **Normalization:** Normalization aims to put all text on a level playing field, e.g., converting all characters to lowercase.
3. **Noise Removal:** Noise Removal cleans up the text, e.g., removing extra whitespaces.

Steps in Text pre-processing is:

● **Remove Extra Whitespaces:**

These are unwanted extra spaces between words. They have no significant meaning; hence it is better to remove them.

- **Remove special characters:**

Special characters typically include any character that is not a letter or number, such as punctuation and whitespace. Removing special characters from a string `result` in a string containing only letters and numbers.

- **Lowercase all texts:** If the text is in the same case, it is easy for a machine to interpret the words because the lower case and uppercase are treated differently by the machine. So, we need to make the text in the same case and the most preferred case is a lowercase to avoid such problems.

- **Remove numbers:** While analysing text for certain applications, the numbers in the string are not of importance. In such cases, we prefer to remove numbers and work on the words only.

- **Remove stop words:** Stop words are the most commonly occurring words in a text which do not provide any valuable information, stop words like they, there, this, where, etc. are some of the stop words. NLTK library is a common library that is used to remove stop words and includes approximately 180 stop words which it removes. If we want to add any new word to a set of words, then it is easy

- **Stemming:** Stemming is a process that reduces words to their root stem, often removing prefixes or suffixes. However, it's not efficient for production and often stems unwanted words. Lemmatization, a technique that uses various stemming algorithms like Porter and Snowball, is widely used in the NLTK library.

- **Lemmatization:** Lemmatization is similar to stemming, used to stem the words into root words but differs in working. Actually, Lemmatization is a systematic way to reduce the words into their lemma by matching them with a language dictionary.

Input:

Step 1: Importing Necessary Packages

```
import re
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
```

```
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('omw-1.4')
```

Step 2: Define Sample Text

sample_text = """ A framework for classifying AGI in levels was proposed in 2023 by Google DeepMind researchers. They define five levels of AGI: emerging, competent, expert, virtuoso, and superhuman. For example, a competent AGI is defined as an AI that outperforms 50% of skilled adults in a wide range of non-physical tasks, and a superhuman AGI is similarly defined but with a threshold of 100%. They consider large language models like ChatGPT or LLaMA 2 to be instances of emerging AGI. """

Step 3: Data Pre-processing

1. Text Lowercasing

```
[7]: text = sample_text.lower()  
     print("Lowercased Text:\n", text)
```

Lowercased Text:

a framework for classifying agi in levels was proposed in 2023 by google deepmind researchers. they define five levels of agi: emerging, competent, expert, virtuoso, and superhuman. for example, a competent agi is defined as an ai that outperforms 50% of skilled adults in a wide range of non-physical tasks, and a superhuman agi is similarly defined but with a threshold of 100%. they consider large language models like chatgpt or llama 2 to be instances of emerging agi.

2. Removing Punctuation

```
[9]: text = re.sub(r'^\w\s', '', text)  
     print("Text after Removing Punctuation:\n", text)
```

Text after Removing Punctuation:

a framework for classifying agi in levels was proposed in 2023 by google deepmind researchers they define five levels of agi emerging competent expert virtuoso and superhuman for example a competent agi is defined as an ai that outperforms 50 of skilled adults in a wide range of nonphysical tasks and a superhuman agi is similarly defined but with a threshold of 100 they consider large language models like chatgpt or llama 2 to be instances of emerging agi

3. Tokenization

```
[11]: tokens = word_tokenize(text)  
      print("Tokenized Text:\n", tokens)
```

Tokenized Text:

['a', 'framework', 'for', 'classifying', 'agi', 'in', 'levels', 'was', 'proposed', 'in', '2023', 'by', 'google', 'deepmind', 'researchers', 'they', 'define', 'five', 'levels', 'of', 'agi', 'emerging', 'competent', 'expert', 'virtuoso', 'and', 'superhuman', 'for', 'example', 'a', 'competent', 'agi', 'is', 'defined', 'as', 'an', 'ai', 'that', 'outperforms', '50', 'of', 'skilled', 'adults', 'in', 'a', 'wide', 'range', 'of', 'nonphysical', 'tasks', 'and', 'a', 'superhuman', 'agi', 'is', 'similarly', 'defined', 'but', 'with', 'a', 'threshold', 'of', '100', 'they', 'consider', 'large', 'language', 'models', 'like', 'chatgpt', 'or', 'llama', '2', 'to', 'be', 'instances', 'of', 'emerging', 'agi']

4. Stop word Removal

```
[13]: stop_words = set(stopwords.words('english'))
      filtered_tokens = [word for word in tokens if word not in stop_words]
      print("Text after Stopword Removal:\n", filtered_tokens)
```

Text after Stopword Removal:

```
['framework', 'classifying', 'agi', 'levels', 'proposed', '2023', 'google', 'deepmind',
'researchers', 'define', 'five', 'levels', 'agi', 'emerging', 'competent', 'expert', 'virtuoso', 'superhuman', 'example', 'competent', 'agi', 'defined', 'ai', 'outperforms', '50', 'skilled', 'adults', 'wide', 'range', 'nonphysical', 'tasks', 'superhuman', 'agi', 'similarly', 'defined', 'threshold', '100', 'consider', 'large', 'language', 'models', 'like', 'chatgpt', 'llama', '2', 'instances', 'emerging', 'agi']
```

5. Lemmatization

```
[15]: lemmatizer = WordNetLemmatizer()
      lemmatized_tokens = [lemmatizer.lemmatize(word) for word in filtered_tokens]
      print("Text after Lemmatization:\n", lemmatized_tokens)
```

Text after Lemmatization:

```
['framework', 'classifying', 'agi', 'level', 'proposed', '2023', 'google', 'deepmind', 'researcher', 'define', 'five', 'level', 'agi', 'emerging', 'competent', 'expert', 'virtuoso', 'superhuman', 'example', 'competent', 'agi', 'defined', 'ai', 'outperforms', '50', 'skilled', 'adult', 'wide', 'range', 'nonphysical', 'task', 'superhuman', 'agi', 'similarly', 'defined', 'threshold', '100', 'consider', 'large', 'language', 'model', 'like', 'chatgpt', 'llama', '2', 'instance', 'emerging', 'agi']
```

6. Consolidating Pre-processing Steps

```
[27]: def preprocess_text(text):
      text = text.lower()
      text = re.sub(r'^\w\s', '', text)
      tokens = word_tokenize(text)
      stop_words = set(stopwords.words('english'))
      filtered_tokens = [word for word in tokens if word not in stop_words]
      lemmatizer = WordNetLemmatizer()
      lemmatized_tokens = [lemmatizer.lemmatize(word) for word in filtered_tokens]
      return ' '.join(lemmatized_tokens)

      processed_text = preprocess_text(sample_text)
      print("Preprocessed Text:\n", processed_text)
```

Preprocessed Text:

```
framework classifying agi level proposed 2023 google deepmind researcher define five level agi emerging competent expert virtuoso superhuman example competent agi defined ai outperforms 50 skilled adult wide range nonphysical task superhuman agi similarly defined threshold 100 consider large language model like chatgpt llama 2 instance emerging agi
```

Conclusion: Thus, we have successfully studied and implemented Data Pre-processing in NLP